

Conceptos avanzados

19. Reescritura de historial

- **git rebase -i** : Abre un editor interactivo que permite modificar, combinar (squash), eliminar o cambiar el orden de una serie de commits pasados.
Se usa para limpiar y organizar tu historial de trabajo local antes de compartirlo con otros, permitiéndote que diez commits de prueba se conviertan en uno solo bien estructurado.
- **git commit --amend** : Modifica el último commit realizado, permitiéndote añadir archivos olvidados o cambiar el mensaje descriptivo.
Se usa cuando acabas de guardar un cambio y notas un error tipográfico o te das cuenta de que te faltó incluir una pequeña modificación, evitando así crear un commit extra innecesario.
- **git filter-branch** : Herramienta pesada que recorre todo el historial del proyecto para aplicar un cambio masivo en cada commit (como por ejemplo borrar un archivo sensible en todas las versiones pasadas).
Se usa únicamente en casos de emergencia o reestructuraciones profundas, ya que reescribe la historia completa y puede causar problemas a otros colaboradores.
- **git replace** : Permite sustituir la visualización de un commit (u objeto) por otro diferente mediante una referencia interna, sin alterar físicamente la base de datos ni los hashes originales del historial, al ejecutarlo Git "engaña" a las herramientas de lectura (como git log) para que proyecten el contenido del reemplazo en lugar del original.
Se utiliza principalmente para conectar historiales independientes (por ejemplo unir un repositorio antiguo con uno nuevo), corregir metadatos erróneos en commits antiguos sin invalidar la cadena de descendientes, o aligerar repositorios masivos sustituyendo objetos pesados por versiones simplificadas sin comprometer la integridad del grafo.

20. Inspección interna

- **git reflog** : Muestra un historial detallado de todos los movimientos que ha hecho el puntero HEAD en tu repositorio local, incluyendo cambios de rama, commits deshechos o rebases fallidos.
Se usa como una "red de seguridad" de última instancia para recuperar commits o ramas que parecen haber desaparecido después de un error técnico.
- **git cat-file** : Permite visualizar el contenido, el tipo o el tamaño de cualquier objeto almacenado internamente en la base de datos de Git (commits, árboles o archivos).
Se usa cuando necesitas inspeccionar técnicamente cómo Git está guardando un dato específico para entender su estructura interna sin necesidad de extraerlo a tu carpeta de trabajo.
- **git hash-object** : Calcula el código único (SHA-1) de un archivo y, opcionalmente, lo guarda directamente en la base de datos de objetos de Git.

Se usa para verificar la integridad de un archivo o para añadir manualmente contenido al repositorio de forma programática sin pasar por el flujo de trabajo estándar de los comandos habituales.

- **git show-ref** : Lista todas las referencias disponibles en el repositorio, mostrando los códigos SHA-1 junto a los nombres de las ramas y etiquetas (tags).
Se usa para verificar rápidamente a qué commit exacto están apuntando tus ramas locales y remotas, asegurando que la sincronización sea precisa.
- **git rev-parse** : Herramienta que traduce nombres amigables (como por ejemplo el nombre de una rama, una etiqueta o palabras como "HEAD") en el código identificador (Hash) real que Git entiende internamente.
Se usa principalmente en la creación de scripts de automatización para asegurarse de que un comando reciba la dirección exacta de un commit, o para verificar rápidamente si una ruta o referencia existe antes de ejecutar una acción compleja.

21. Integridad y mantenimiento

- **git fsck** : Realiza una auditoría completa del sistema de archivos de Git para verificar la integridad de todos los objetos y buscar archivos que hayan quedado huérfanos o corruptos.
Se usa cuando sospechas que el repositorio está dañado tras un apagón o un error de disco, sirviendo como una herramienta de diagnóstico para asegurar que toda tu historia de cambios es válida y recuperable.
- **git gc** : Ejecuta el "Garbage Collector" (recolector de basura) de Git, que limpia archivos temporales innecesarios, comprime los objetos de la base de datos y optimiza el almacenamiento local.
Se usa periódicamente en proyectos muy grandes o con mucho movimiento para reducir el peso de la carpeta ".git" y acelerar el rendimiento de comandos como git status o git log.

22. Entornos de trabajo paralelos

- **git worktree** : Herramienta que permite gestionar múltiples directorios de trabajo vinculados a un mismo repositorio compartido.
Se usa cuando necesitas trabajar en varias ramas al mismo tiempo sin tener que andar cambiando de una a otra (haciendo checkout) en una sola carpeta, permitiéndote, por ejemplo, mantener abierta la rama de "producción" en una carpeta mientras desarrollas una "nueva función" en otra carpeta distinta.
- **git worktree add** : Crea un nuevo directorio de trabajo vinculado al mismo repositorio, permitiéndote tener dos ramas diferentes abiertas y editables al mismo tiempo en carpetas separadas.
Se usa cuando estás a mitad de una tarea compleja y necesitas cambiar urgentemente a otra rama para corregir un error, sin tener que hacer stash o commit de lo que estás haciendo.
- **git worktree list** : Muestra una lista detallada de todos los directorios de trabajo auxiliares que tienes activos actualmente y a qué rama está dedicado cada uno.

Se usa para llevar un control de tus "talleres" abiertos y evitar confusiones o conflictos al trabajar con múltiples versiones del mismo proyecto en paralelo.

- **git worktree remove** : Elimina de forma segura un directorio de trabajo auxiliar y limpia las referencias internas de Git asociadas a esa carpeta.
Se usa una vez que has terminado la tarea paralela y ya no necesitas ese espacio de trabajo extra, manteniendo tu entorno de desarrollo organizado y libre de carpetas temporales.

23. Submódulos

- **git submodule** : Herramienta que permite insertar y gestionar un repositorio de Git completo dentro de otro repositorio de Git (como una carpeta que es, a su vez, otro proyecto independiente).
Se usa cuando tu proyecto depende de una librería o herramienta externa que tiene su propio historial y desarrollo, permitiéndote mantener ambos proyectos vinculados pero separados.
- **git submodule add** : Descarga un repositorio externo y lo registra como un componente oficial dentro de una carpeta de tu proyecto principal.
Se usa cuando decides que vas a utilizar el código de otra persona o equipo como una pieza fija de tu software, estableciendo una conexión que permite traer actualizaciones de ese código específico en el futuro.
- **git submodule init** : Prepara los archivos de configuración local para que Git reconozca los submódulos que ya existen en un proyecto que acabas de descargar.
Se usa la primera vez que clonas un repositorio que contiene otros proyectos dentro, "avisándole" a tu sistema que debe estar listo para activar esas carpetas externas.
- **git submodule update** : Descarga el contenido real y los archivos de los submódulos registrados, sincronizándolos con la versión exacta que requiere el proyecto principal.
Se usa inmediatamente después del "init" o cuando un compañero ha actualizado la versión de la librería externa y necesitas que tu computadora descargue esos nuevos cambios para que todo funcione correctamente.

24. Depuración y búsqueda de errores

- **git bisect** : Herramienta de búsqueda binaria que divide el historial de commits a la mitad sucesivamente para encontrar exactamente cuál introdujo un error.
Se usa cuando el código se rompió en algún momento del pasado y tienes demasiados cambios acumulados como para revisar uno por uno manualmente.
- **git bisect start** : Inicia el modo de depuración interactiva de Git y prepara al sistema para empezar a marcar puntos en el historial.
Se usa como el primer paso obligatorio cuando decides que vas a realizar una revisión de errores para aislar el fallo entre dos versiones del proyecto.
- **git bisect good** : Informa a Git que la versión actual del código (o una versión antigua específica) funciona correctamente y no tiene el error.

Se usa para marcar el punto de partida "sano" en el pasado o para confirmar que, tras la última división del historial, el error aún no aparecía en ese punto.

- **git bisect bad** : Informa a Git que la versión actual tiene el fallo o comportamiento inesperado que estás buscando. Se usa al principio para marcar el commit que sabes que falla y, durante el proceso, para decirle a Git que el error sigue presente en la mitad de la historia que estás probando.
- **git bisect reset** : Finaliza la búsqueda binaria, limpiando todos los estados temporales y te devuelve automáticamente a la rama y al commit donde estabas antes de empezar a buscar. Se usa una vez que has identificado el commit culpable decides abandonar la búsqueda para volver a tu trabajo normal.

25. Estructura interna de Git

- **.git/objects** : Es el "corazón" o base de datos donde Git almacena el contenido real de todos tus archivos, commits y árboles de directorios, comprimidos y protegidos mediante códigos únicos (hashes).
Se usa internamente para guardar cada versión de tu proyecto de forma eficiente, permitiendo que Git recupere cualquier estado pasado del código sin importar cuánto tiempo haya pasado.
- **.git/refs** : Carpeta que contiene punteros o "marcadores" hacia los commits específicos donde terminan tus ramas y etiquetas (tags).
Se usa para que Git sepa exactamente en qué commit termina la rama main, la rama desarrollo o dónde se encuentra una versión específica (como la v1.0), sirviendo como el mapa de rutas del repositorio.
- **.git/index** : Archivo binario que actúa como el Staging Area (zona de preparación), guardando la lista de todos los cambios que has marcado con "git add" pero que aún no has guardado definitivamente.
Se usa como un "puente" crítico entre tu carpeta de trabajo y el próximo commit, permitiendo que Git sepa exactamente qué piezas de código debe empaquetar en la siguiente versión.
- **.git/HEAD** : Archivo de texto simple que indica qué referencia (rama o commit) tienes activa en este preciso momento en tu carpeta de trabajo.
Se usa como el indicador de "usted está aquí", permitiendo que Git actualice los archivos correctos cuando cambias de rama o cuando realizas un nuevo guardado de información.

26. Exportación y comparación

- **git archive** : Genera un archivo comprimido (como un .zip o .tar) que contiene únicamente los archivos de una rama o commit específico, omitiendo toda la carpeta oculta .git.
Se usa cuando necesitas enviar el código fuente a alguien que no usa Git o para subir una versión limpia del proyecto a un servidor de producción sin arrastrar el historial de cambios.

- **git diff** : Muestra las diferencias línea por línea entre los cambios que tienes en tu carpeta de trabajo y lo que ya está guardado en el último commit.
Se usa para revisar exactamente qué has escrito o borrado antes de decidir si esos cambios están listos para ser preparados con un add.
- **git diff HASH1 HASH2** : Compara directamente el contenido de dos puntos diferentes en el historial del proyecto utilizando sus códigos identificadores
Se usa cuando necesitas saber qué cambió exactamente entre dos versiones distantes, como por ejemplo, qué código se añadió entre la versión de hace un mes y la de hoy.
- **git merge-base** : Encuentra el "ancestro común" más cercano entre dos ramas diferentes, es decir, el último punto en el tiempo donde ambas compartían el mismo código.
Se usa principalmente en tareas de depuración o automatización para entender en qué momento exacto se separaron dos líneas de trabajo antes de intentar unirlos.
- **git reset -p** : Abre un modo interactivo que permite elegir, fragmento por fragmento, qué cambios quieres sacar del área de preparación (staging) para devolverlos al directorio de trabajo.
Se usa cuando te arrepientes de haber añadido ciertas líneas de código con "git add" y quieres "des-prepararlas" de forma quirúrgica sin afectar al resto de los archivos que sí quieres guardar.